

Competitive Programming

Section 50: Advanced Algorithms Mixed Contest

Competitive Programming Course

2026-07-10

Table of contents

1	Mixed advanced contest habits	3
1.1	DAG Grundy	3
1.2	General CRT	3
1.3	Fast Fibonacci	4
1.4	Contest lesson	4
1.5	CRT worked example	4
1.6	Fast doubling trace	5
1.7	Mixed-contest debugging	5
2	Exercises	6
3	Mixed-contest proof checkpoints	7

1 Mixed advanced contest habits

By now, the hard part is often recognizing the model: impartial game on a DAG, congruence merging, or fast recurrence evaluation. The local tasks are small but force you to state the invariant precisely before coding.

1.1 DAG Grundy

For impartial games on a DAG, every state has a Grundy number:

```
grundy[v] = mex({ grundy[to] for every move v -> to })
```

`mex` is the minimum excluded nonnegative integer. A state is winning exactly when `grundy[v] != 0`.

Because the graph is acyclic, compute states in reverse topological order. If vertices are already numbered so edges go forward, iterate from `n` down to 1; otherwise explicitly topologically sort first.

1.2 General CRT

To merge:

```
x = a mod m
x = b mod n
```

Let $g = \gcd(m, n)$. A solution exists iff:

```
(b - a) % g == 0
```

Reduce:

```
m' = m/g, n' = n/g
```

Find t satisfying:

```
a + m*t = b mod n
m*t = b-a mod n
m' * t = (b-a)/g mod n'
```

Since m' and n' are coprime, invert $m' \bmod n'$. The new modulus is $\text{lcm}(m, n) = m/g * n$. Normalize the answer into $[0, \text{lcm})$.

1.3 Fast Fibonacci

Fast doubling computes $(F(n), F(n+1))$ recursively:

```
c = F(2k)    = F(k) * (2*F(k+1) - F(k))
d = F(2k+1)  = F(k)^2 + F(k+1)^2
```

If n is even, return (c, d) . If n is odd, return $(d, c+d)$.

This is a matrix-exponentiation result specialized to Fibonacci, with fewer constant factors and less code.

1.4 Contest lesson

These three problems look unrelated. The common skill is compression:

- a whole game state becomes one Grundy number;
- two congruences become one congruence;
- a recurrence query becomes two values $(F(n), F(n+1))$.

1.5 CRT worked example

Merge:

```
x = 2 mod 6
x = 5 mod 9
```

$\gcd(6,9)=3$. Since $5-2=3$ is divisible by 3, a solution exists.

Check values:

```
2, 8, 14, 20, 26, ...
```

Modulo 9, these are:

```
2, 8, 5, ...
```

So $x = 14$ works. The merged modulus is:

```
lcm(6,9) = 18
```

All solutions are:

```
x = 14 mod 18
```

If the residues had disagreed modulo 3, there would be no solution.

1.6 Fast doubling trace

To compute $F(5)$, recurse to $F(2)$, then $F(1)$, then $F(0)$.

At $k = 1$:

$$\begin{aligned}F(2) &= F(1) * (2 * F(2) - F(1)) = 1 \\F(3) &= F(1)^2 + F(2)^2 = 2\end{aligned}$$

At $k = 2$:

$$\begin{aligned}F(4) &= F(2) * (2 * F(3) - F(2)) = 3 \\F(5) &= F(2)^2 + F(3)^2 = 5\end{aligned}$$

The recursive function returns pairs so every level has exactly the two values needed by the next doubling step.

1.7 Mixed-contest debugging

For Grundy, test terminal vertices and vertices whose outgoing Grundy set is $\{1\}$ so the mex is 0. For CRT, test incompatible residues:

$$\begin{aligned}x &= 0 \pmod{2} \\x &= 1 \pmod{4}\end{aligned}$$

For Fibonacci, test:

$$F(0)=0, F(1)=1, F(2)=1, F(10)=55$$

These anchors catch most implementation mistakes quickly.

2 Exercises

3 Mixed-contest proof checkpoints

For DAG Grundy, a state is winning iff some successor has Grundy value zero after mex logic; process reverse topological order. CRT needs compatibility $a-b$ divisible by $\gcd(m,n)$ before constructing a solution with extended Euclid. Fast doubling follows directly from the two Fibonacci addition identities. Test terminal vertices, incompatible moduli, and $F(0)$.

- A. DAG Grundy: For each start vertex in a DAG, print whether the first player wins.
- B. General CRT: For each pair of congruences, print the smallest nonnegative solution and lcm modulus, or NO.
- C. Fast Fibonacci: Answer Fibonacci queries modulo $1e9+7$.